PAOO - L02 - prj

Contents

Introducere în dezvoltarea jocurilor 1

Etape realizare proiect 9

Etapa 1 - prezentare proiect: referat 9

Etapa 2 – implementarea proiectului 10

Câteva observații despre șablonul “main loop” 10

Descrierea scheletului proiectului 11

Diagramă proiect 11

Descriere clase proiect 12

Anexa 1. 15

Introducere în dezvoltarea jocurilor

Jocurile și aplicațiile grafice sunt prezente în permanență în jurul nostru, fie ca este o aplicație de mobil, pe calculator sau chiar în browser-ul de internet.

Dezvoltare de jocuri și aplicații grafice are câteva particularități, care o fac atat dificilă și complexă, dar în același timp foarte interesantă, având întotdeauna un rezultat grafic, care poate fi vizualizat și cu care de cele mai multe ori se poate interacționa.

Jocurile sunt o categorie aparte a aplicațiilor grafice, folosite cel mai adesea pentru divertisment. Reducând la esență, jocurile pot fi definite ca o formă structurată și organizată de activitate cu un set de reguli în care se iau decizii pentru a atinge unul sau mai multe obiective sau un scop final.

Există multe tipuri de jocuri, care adăuga diverse nivele de complexitate, cum ar fi:

- **Jocuri** simple de **decizie**: necesită ca jucătorul să ia anumite decizii.
- **Competiție**: adaugă un nivel de complexitate prin implicarea mai multor jucători aflați în competiție.
- **Puzzle**: adaugă o problemă de rezolvat.
- **Sisteme interactive**: forme mai complexe de joacă care presupun seturi de reguli și interacțiuni complexe.

Un joc sau o aplicație video este un sistem software care rulează într-o buclă infinită, generand imagini pe ecran cu o frecvență suficient de mare astfel încât să nu fie percepută tranziția de către ochiul uman. Aceasta frecvență trebuie sa fie în general peste 25 de cadre pe secundă, pentru a avea o animație sau tranziții fluente.

Realizarea de jocuri și aplicații grafice este deosebit de complexă, procesul de dezvoltare de jocuri având multe elemente componente:

- **Proiectarea contextului**: implică definirea unui gen de joc, a unei tematici și a unei povești incipiente care reprezintă intriga pe care se bazează jocul (desen realizat de mana sau cu un editor grafic)

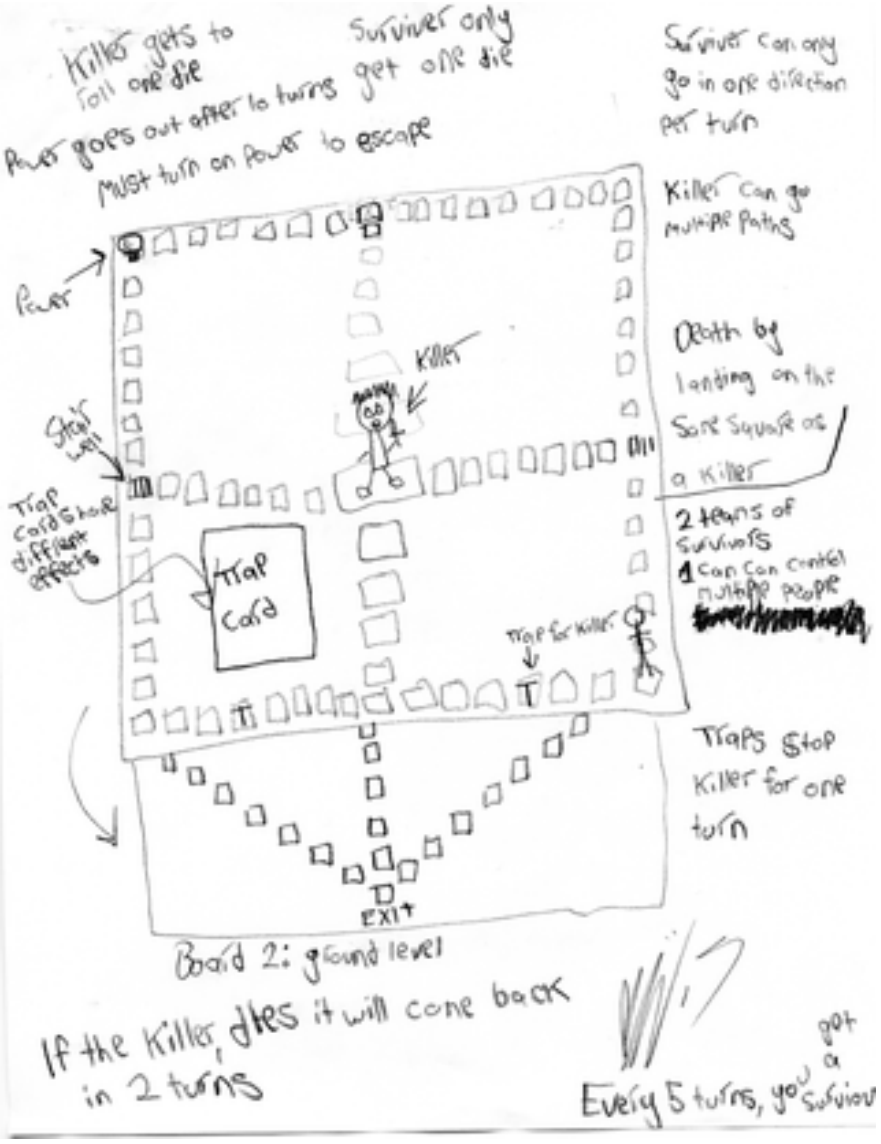


Fig. 1 - Proiectarea contextului

- **Proiectarea sistemului**: implică definirea regulilor principale care guvernează modul în care se poate juca sau interacționa cu jocul. Mai are și denumirea de Gameplay sau mecanici de joc (Game Mechanics), și descriu cum pornește jocul, cum se joacă, cum se progresează și cum se câștigă.

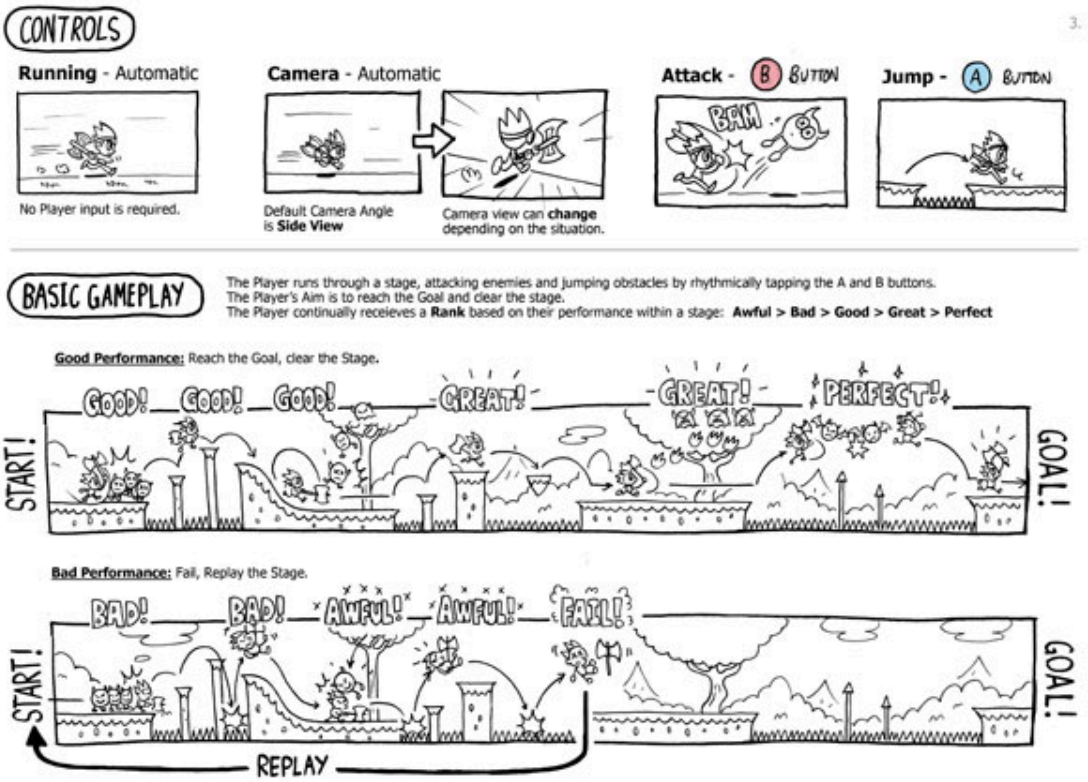


Fig. 2 - Proiectarea sistemului

- **Proiectarea conținutului**: implică definirea caracterelor, animațiilor, a obiectelor, a interacţiunilor şi a parcursului pe care îl poate avea un jucător.

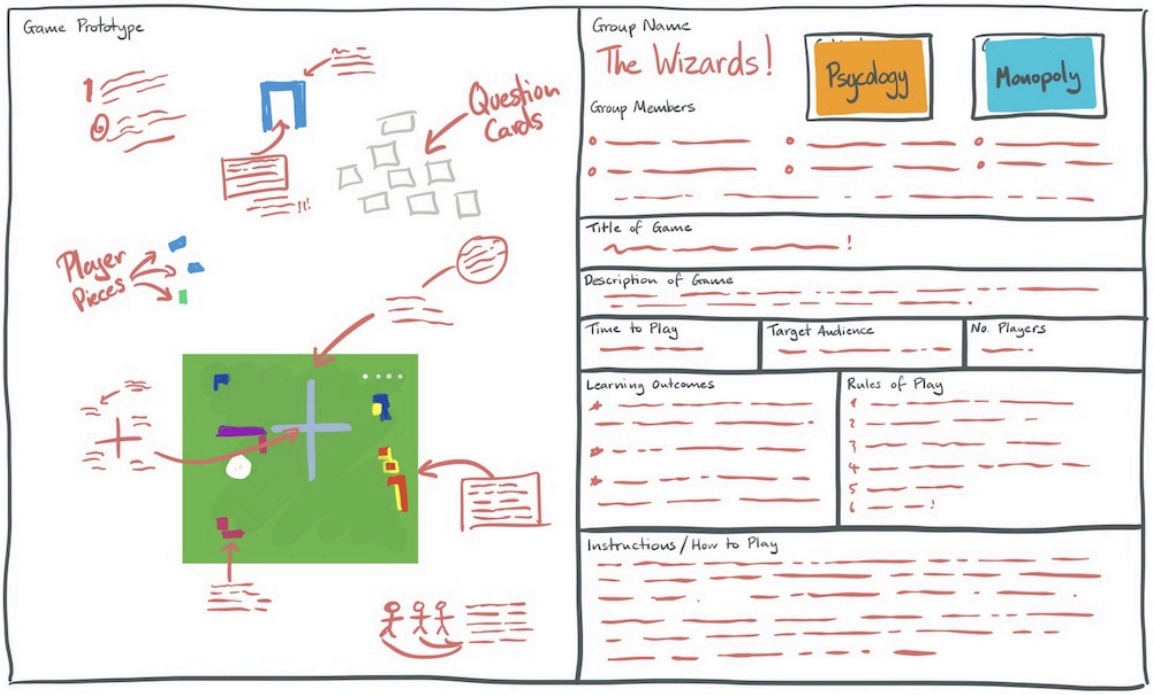


Fig. 3 - Proiectarea conținutului

- **Proiectarea nivelurilor**: implică definirea hărții de joc, a poziției caracterelor și a obiectelor în scenă, dar și dacă jocul are mai multe etape sau nivele pe care le poate accesa jucătorul în anumite condiții.

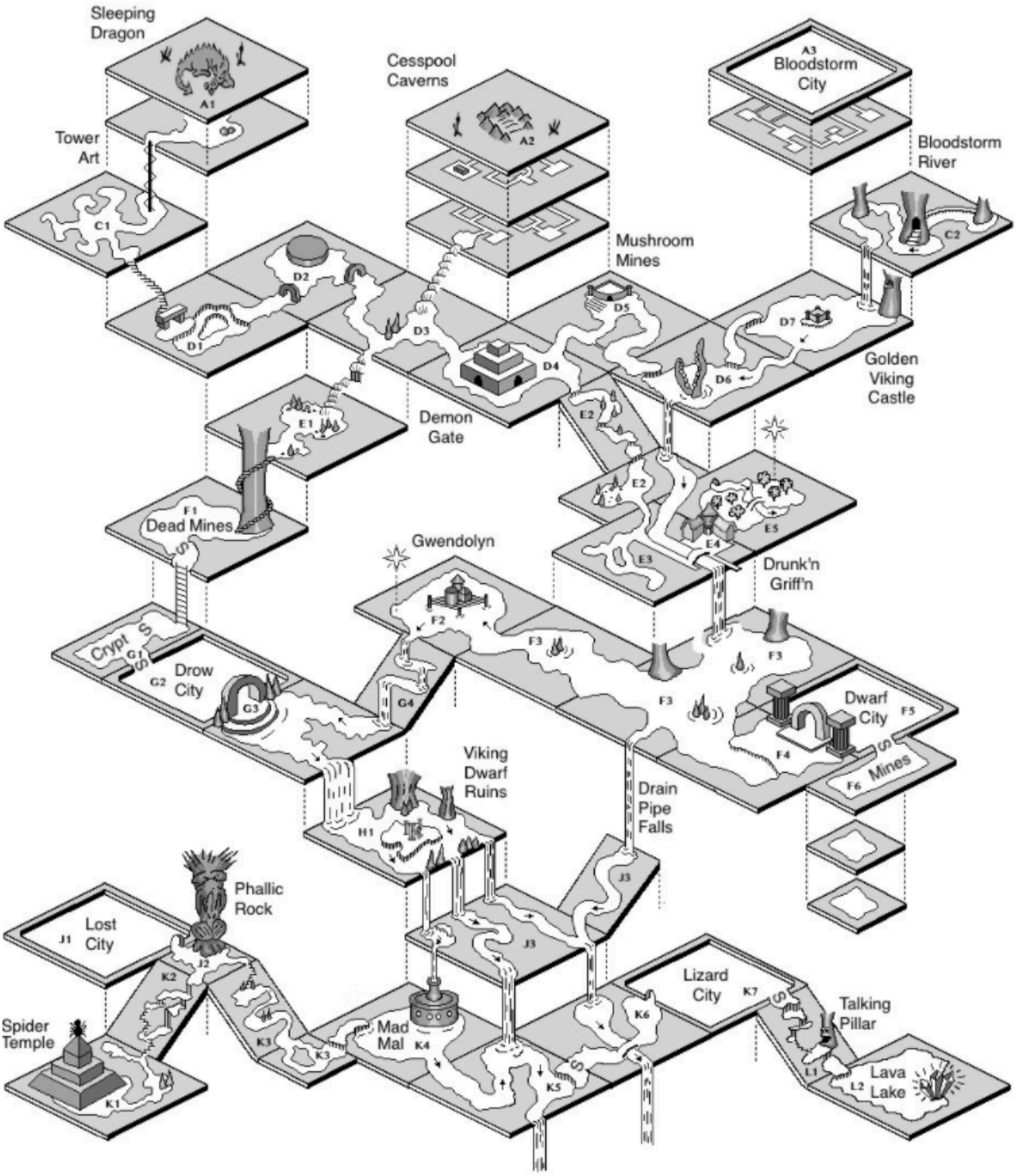


Fig. 4 - Proiectarea nivelurilor

- **Proiectarea interfeței cu utilizatorul**: implică definirea mediului de joc, a interfeței din cadrul jocului cât și a interfeței interacțiunilor pe care le poate avea jucătorul cu diverse obiecte din joc.

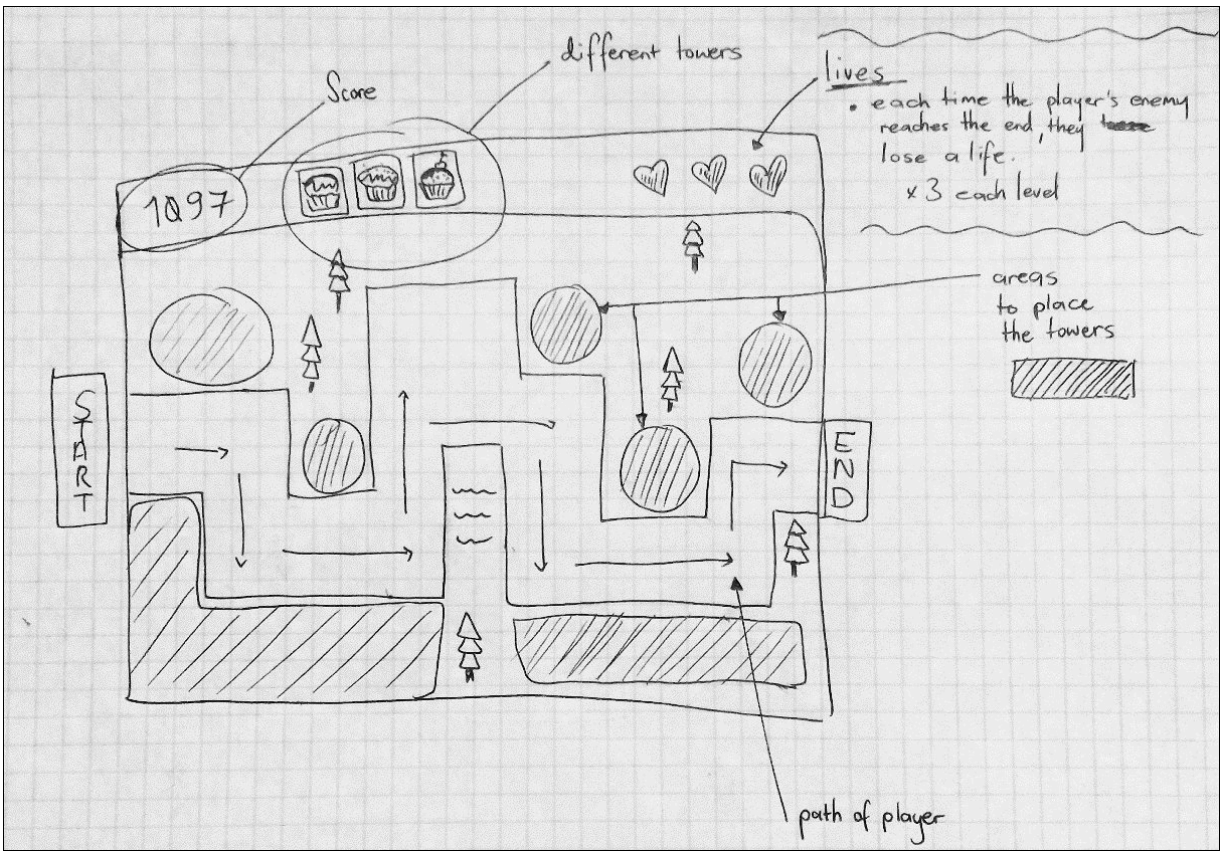


Fig. 5 - Proiectarea interfeței cu utilizatorul

Mai mult, jocurile și aplicațiile grafice sunt sisteme complexe atât la nivel arhitectural cat și conceptual. Astfel, un joc tip conține foarte multe componente cum ar fi:

- **Rendering**: Sisteme de redare a imaginilor, care asigura încărcarea, procesarea și afișarea pe ecran a scenelor. Sunt incluse sisteme de redare a terenului, a obiectelor, simularea apei, afișarea de text, texturare etc.

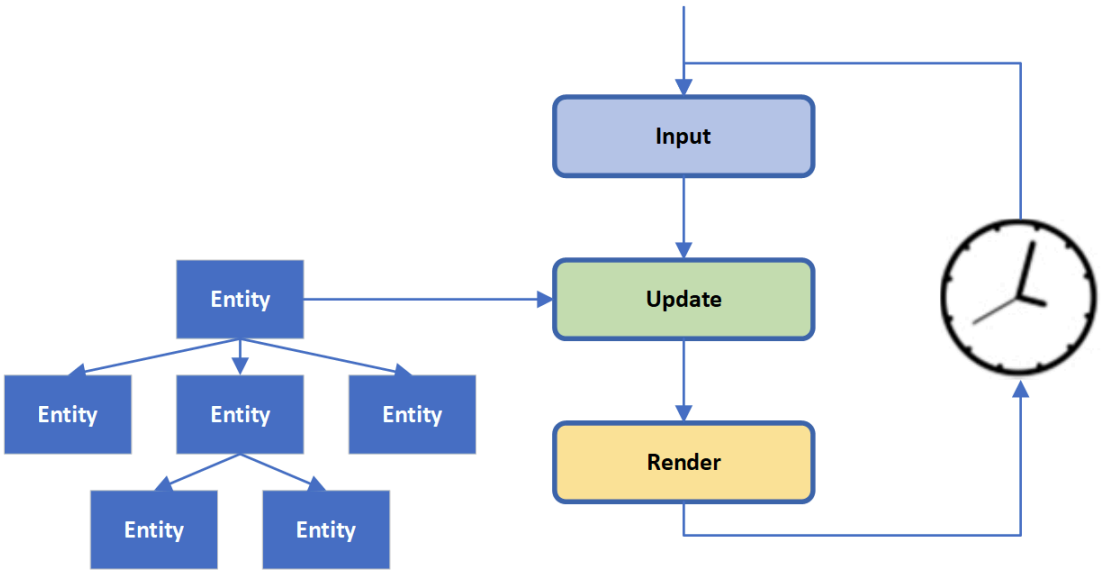


Fig. 6 - Randare

- **Animatii**: Sisteme care realizează și calculează animațiile obiectelor din scena, în general pe baza unui schelet.

- **Managementul scenei**: Sisteme de gestiune a scenei și a obiectelor din scenă.

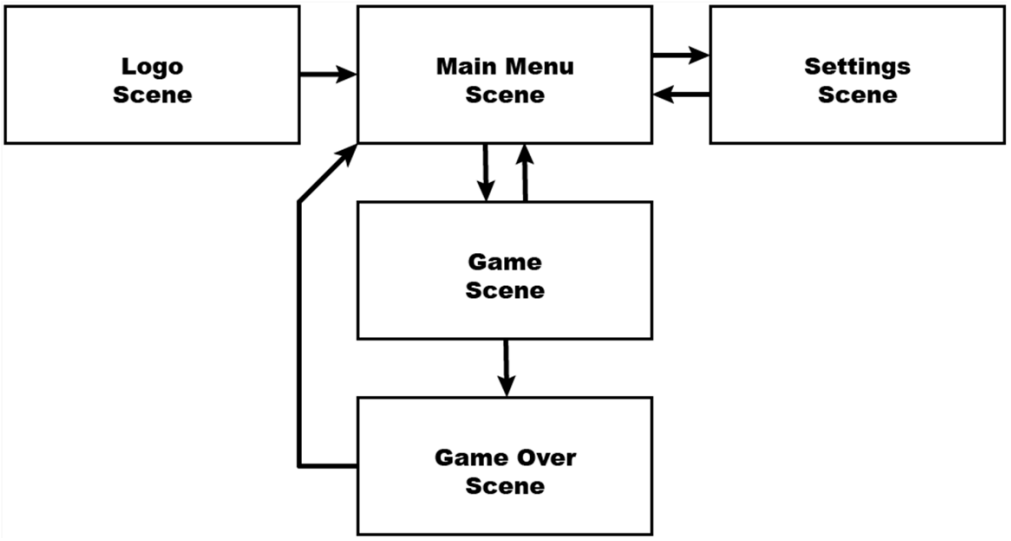


Fig. 7 - Managementul scenei

- **Asset management**: Sisteme de gestiune a resurselor necesare. Sunt incluse sisteme de încărcare, citire și înțelegere a diverselor formate de fișiere.

- **Camera**: Modul de afișare și controlul perspectivei asupra jocului.



Fig. 8 - Camera

- **Multiplayer**: Sistem de partajare sau de joc multi-utilizator folosind același dispozitiv sau în retea sau internet.

- **GUI**: Graphical User Interface – Sisteme de meniuri, butoane și alte elemente de interacțiune grafica cu utilizatorul.

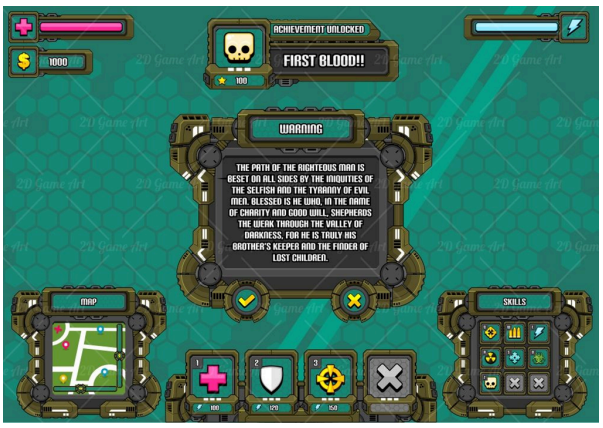


Fig. 9 - GUI

- **Input**: Sisteme de captare a inputului de la diverse dispozitive: mouse, tastatură

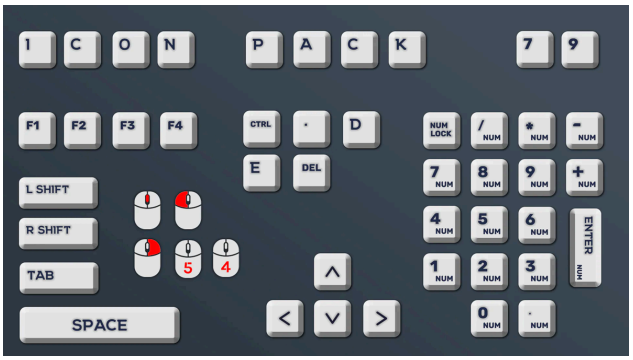


Fig. 9 - Input

- **Pseudo AI**: Sistem de inteligență artificială.

Procesul de dezvoltare a unui joc include mai mulți pași, prezentați și în figura de mai jos:

- **Pre-producție**: include printre altele realizarea conceptului, a documentului de proiectare (GDD - Game Design Document), a planului de proiect și a unui prototip. Acestea sunt deosebit de importante, pentru că pe baza lor se va evalua fezabilitatea proiectului. De exemplu conceptul poate fi bun, dar dacă tehnologic nu este posibila realizarea unui prototip nu este fezabil. Sau poate planul de implementare este prea lung și nu exista fonduri suficiente pentru alocare etc.
- **Producție**: include faza principală de proiectare și producție. Producția include atât dezvoltarea aplicației, la nivel de programare), cât și a resurselor grafice și audio necesare. După aceasta fază ar trebui sa existe un produs testat, gata de publicare
- **Post-producție**: fază deosebit de importantă pentru retenția de clienți și atragerea de noi clienți, prin marketing, mentenanță, furnizarea de conținut nou și atractiv, rezolvarea problemelor, crearea unei comunități etc.

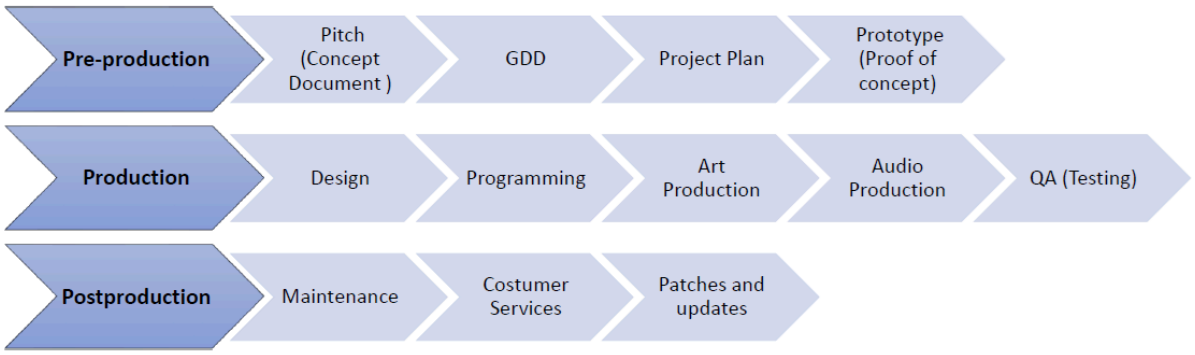


Fig. 10 - Procesul de dezvoltare a unui joc

Pe scurt, **etapele creării unui joc cuprind:**

- Alegerea tipului de joc.
- Crearea poveștii jocului (povestea ce va sta la baza jocului + mecanică aferentă jocului).
- Crearea story boardului - se desenează fiecare scenă a întregii povești.
- Schițarea caracterelor.
- Crearea personajelor și a obiectelor ce vor fi folosite in joc. Texturarea modelelor.
- Animarea elementelor dinamice (mers etc.).
- Crearea efectelor speciale (explozie etc.).
- Importarea sunetelor necesare.
- Architectural Design Document.
- Detailed Design Document.
- Programarea propriu-zisă a jocului.
- Testare - optimizare.

Resurse:

- OpenGameArt: https://opengameart.org/
- LetsMakeGames: https://letsmakegames.org/resources/art-assets-for-game-developers/
- PixelProspector: https://pixelprospector.com/

Cum să-ți creezi propriile elemente:

- Asprite: https://www.aseprite.org/
- PixelEdit: https://pyxeledit.com/
- GraphicsGale: https://graphicsgale.com/us/
- GIMP: https://www.gimp.org/
- Inkscape: https://inkscape.org/

Asigurați-vă că dalele se potrivesc împreună și adăugați variații pentru a le face mai interesante!

Etape realizare proiect

Jocul, implementat în Java de către o echipă de doi studenți din aceeași grupă, va avea ca scop parcurgerea mai multor niveluri (**TREI**) pentru a atinge un anumit țel (nivelul final, o luptă cu personajul negativ principal, etc). Personajul sau personajele principale vor avea o poveste și vor realiza o serie de misiuni pentru a atinge un anumit scop. Programul va trebui să poată salva starea jocului la un moment dat (**SAVE**), să încarce jocul din starea anterioară (**LOAD**), să calculeze un anumit scor și să țină evidența celor mai bune scoruri și a celor mai buni jucători din toate timpurile.

În implementare se vor folosi **șabloane de proiectare**, **baze de date**, **tratarea excepțiilor -** implementarea unor clase specifice pt această operație. **Codul va trebui să fie comentat în vederea realizării documentației** (Javadoc, Doxygen, etc.).

Propunerea de proiect (etapa de preproductie)

Prima etapă constă în planificarea și proiectarea aplicației. Pentru aceasta etapă se va realiza un document care va conține:

- **Proiectarea contextului și descriere detaliată**
- **O listă a sarcinilor săptămânale ale fiecărui membru al echipei -> GitHub Projects (task-urile vor fi corelate cu commit-urile de pe Git)**
- **Proiectarea sistemului și descriere detaliată -> Drawio / STARuml**
    - **integrarea a minim 2 sabloane de proiectare (de exemplu, Singleton pentru gestionarea resurselor sau Factory pentru crearea inamicilor)**
    - **corelarea sabloanelor de proiectare cu diagrama de proiect**
- **Proiectarea conținutului și descriere detaliată**
- **Proiectarea nivelurilor și descriere detaliată -> Tiled Map Editor**
    - **Proiectul trebuie să aibă minim 3 niveluri**
    - **Minim 2 game sprite-uri ce vor fi utilizate**
- **Proiectarea interfeței cu utilizatorul și descriere detaliată -> Figma / Canva**
- **Resursele biografice utilizate bibliografie in formatul IEEE**

În **Anexa 1** sunt indicate elementele ce intră în punctarea propunerii de proiect.
Aceasta etapa presupune ca în **săptămâna 7** să fie urcat pe GitHub un pdf care să conțină informațiile cerute. Fișierul va avea numele de forma:

NrGrupa_NumePrenume1_NumePrenume2.pdf
Exemplu: 1220B_IonescuDaniel_PopesculIon.pdf

Echipele vor gestiona progresul și versionarea proiectului pe GitHub. **Începând cu saptămâna 8 fiecare membru al echipei va avea cel puțin un commit pe saptămână conform împărțirii sarcinilor din propunerea de proiect.**
După stabilirea componenței, un membru va accesa invitația și va crea echipa, iar cel de-al 2-lea membru va adera la echipa deja creată. Astfel, fiecare echipă va avea propriul repo cu scheletul de proiect ca starter code.

Link invitație GitHub: https://classroom.github.com/a/OYx22Cxf

Implementarea proiectului

În fiecare săptămână studenții vor trebui să lucreze la laborator și acasă minim câte o nouă cerință legată de proiect. Astfel, cerintele care trebuie finalizate spre a putea fi verificate (demo cu functionalitatea; intrebari & raspunsuri despre implementare) in fiecare saptamana sunt:

- în săptămâna 8: încărcarea hărții pentru nivelul 1, meniul principal
- în săptămâna 9: implementarea deplasării personajului principal, camera
- în săptămâna 10: adăugarea celorlalte niveluri, inamici, NPC
- în săptămâna 11: tratarea coliziunilor
- în săptămâna 12: salvarea și încărcarea jocului dintr-o bază de date
- în săptămâna 13: finisarea, verificarea elementelor de tip șablon, baza de date, tratarea excepțiilor, adăugarea altor NPC (dacă este cazul), fog of war
- în săptămâna 14: prezentarea proiectului

Câteva observații despre șablonul “main loop”

În momentul implementării o problemă deosebită va fi “refresh-ul” imaginii de pe ecran pentru a putea reactualiza poziția eroilor și obiectelor în scenă, adaptate la comenzile efectuate. Una dintre cele mai utilizate metode de rezolvare este utilizarea unui șablon de proiectare al aplicației care sa poată să redea corect, în fiecare moment, imaginea pe ecran.
O buclă dintr-un joc trebuie să ruleze continuu în timpul acțiunilor acestuia. La fiecare nouă acțiune (tură de joc, dacă este un joc turn base) este necesară procesarea intrărilor utilizatorului (apăsări de taste, mouse, etc) fără blocarea, întreruperea jocului sau pauze, actualizarea stării jocului și redarea consecințelor acțiunilor utilizatorului (noua stare a jocului).
Un astfel de model decuplează progresia timpului de joc de intrările date de utilizator și viteza procesorului.
Bucla jocului (“main loop”) este procesul principal al tuturor firelor de redare a jocului. Este prezent în toate jocurile moderne. Acesta conduce procesul de intrare, actualizarea stărilor interne, randarea, AI și toate celelalte procese (vezi fig. 6)
Rolul funcției sleep este de a ne asigura că jocul nu rulează prea repede, lăsând timp tuturor elementelor să se actualizeze. Această etapă poate lipsi dacă jocul nu necesită o astfel de întârziere. Un mod de a descrie acest șablon se regăsește mai jos (https://gameprogrammingpatterns.com/game-loop.html):

```
while (true)
{
  double start = getCurrentTime();
  processInput();
  update();
  render();

  sleep(start + MS_PER_FRAME - getCurrentTime());
}
```

```
MS_PER_FRAME = milisecunde pe fiecare frame
```

Lămuriri suplimentare se regăsesc la adresele:

https://thenumb.at/cpp-course/design/design/games.html
https://gameprogrammingpatterns.com/game-loop.html
Mai jos este prezentat schematic un schelet de proiect ce va fi pus la dispoziția dvs ca suport de lucru.

Descrierea scheletului proiectului

Diagramă proiect

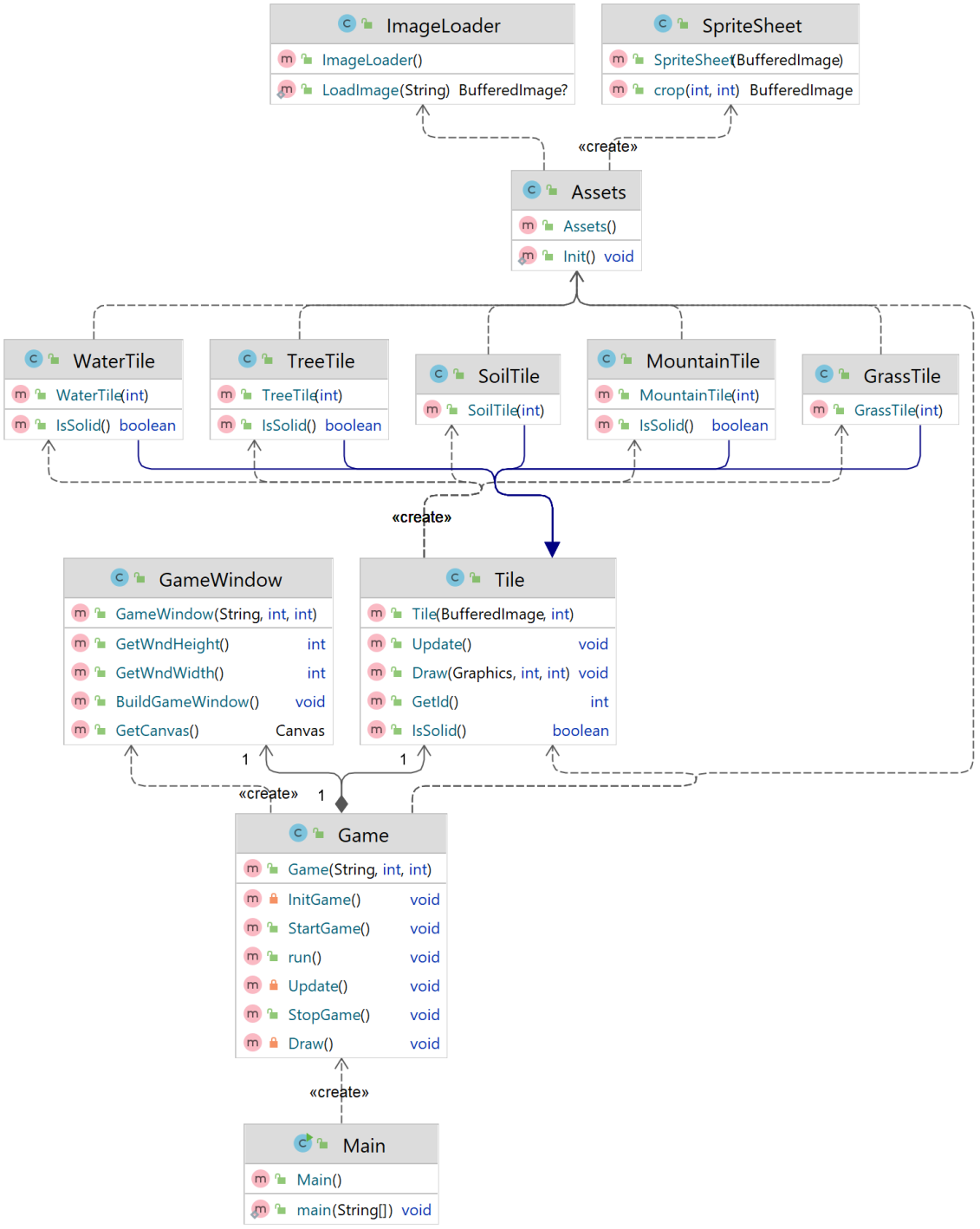


Figura 11. Diagrama UML de clase

În figura de mai sus se poate observa diagrama de clase UML (sunt vizibile doar constructorii și metodele claselor pentru a nu aglomera diagrama). Fiecare clasa are rolul ei în cadrul proiectului și la rândul său instanțiază/moștenesc alte clase.

După cum s-a discutat în laboratorul anterior, trebuie să existe cel puțin o metoda main în una din clase pentru a fi apelată la rularea programului (doar o metoda main poate fi rulata la un moment dat). Așadar, execuția programului începe din clasa Main, metoda main. În cele ce urmează sunt prezentate toate clasele, începând cu Main, precum și firul logic al acțiunilor.

Descriere clase proiect

Main

După cum se poate observa din diagrama prezentată mai sus, această clasă creează o instanță de tipul *Game*, iar mai apoi pornește fluxul activităților prin apelul metodei *StartGame()* din *Game*.

Game

Rolul acestei clase este de a face inițializările (de exemplu de a crea instanțe pentru jucători/inamici, pentru a inițializa harta, etc.), dar și de a menține interfața grafică la zi cu cea ce se întâmplă în joc. Această clasă implementează interfața *Runnable* pentru a avea comportamentul unui fir de execuție (thread). În momentul apelului metodei *StartGame()* se instanțiază un obiect de tip *Thread* pe baza instanței curente a clasei *Game*.

Orice obiect de tip *Thread* trebuie să implementeze metoda *run()* care este apelată atunci când firul de execuție este pornit (*start()*). Această metodă *run()* inițializează jocul prin crearea unei instanțe *GameView*, iar mai apoi controlează numărul de cadre pe secundă printr-o buclă *while* și “pregătește” noua scenă (*Update()*) pe care o va desena pe interfața grafică (*Draw()*).

Metoda *Update()* actualizează starea jocului (de exemplu: modifica poziția jucătorilor pe baza tastelor apăsate, schimbă poziția inamicilor folosind chiar tehnici de inteligență artificială, crează diferite *tile-uri* (dale), etc. Dacă ne referim la un joc 2D care are harta pătrată, aceasta este împărțită în celule de aceeași dimensiune(ca o tablă de sha), iar aceste celule sunt *tile-uri* care pot avea texturi diferite, pot reprezenta diverse obiecte (de exemplu proiectilele unui tanc, acesta poate fi un *tile* ca și restul elementelor de pe hartă dar care are fundalul transparent).

Metoda *Draw()* va desena pe interfața grafică modificările făcute de metoda *Update()*. Interfața grafică este un *canvas*, făcând o analogie cu realitatea, poate fi considerată o pânză pentru desen, pe care sunt desenate diverse obiecte.

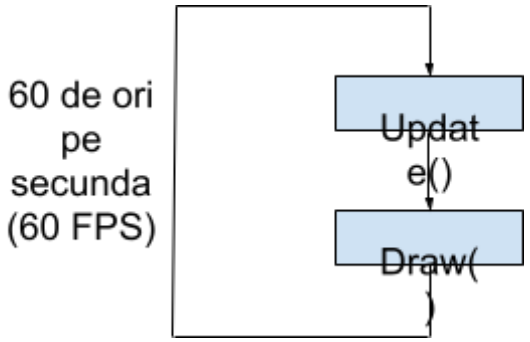


Figura 3. Bucla metodei *run()* din clasa Game

GameWindow

După cum discutam mai sus, metoda *run()* a clasei *Game* crează o instanță a clasei *GameWindow*. Această clasă este responsabilă cu fereastra în care vor fi desenate obiectele pe un *canvas* (se poate observa ca avem o metoda *GetCanvas* care întoarce canvasul). De asemenea, avem un obiect *JFrame* care permite desenarea de butoane, controale, textbox-uri, etc, dar poate conține și un canvas în care pot fi desenate diverse obiecte folosind texturi. Dacă de exemplu dorim să avem un meniu pentru joc, atunci sunt necesare mai multe obiecte *JFrame*, unul pentru meniu în care sunt desenate butoanele, etc și unul pentru acțiunea jocului, iar în funcție de interacțiunea utilizatorului acestea vor fi schimbate între ele.

Tile

Această clasă reține informații despre *tile-urile* (dalele) din joc. În clasa *Game* există o instanță a clasei *Tile*, deoarece clasa *Tile* conține un vector cu obiecte tot de tipul *Tile*, așadar în acest vector vor fi stocate toate „dalele/tile-urile„ din joc în așa fel încat acest vector poate fi parcurs și pentru fiecare element se apelează metoda *Update()* și apoi *Draw()*. De asemenea, clasa *Tile* mai reține și câte o instanță pentru fiecare sub-tip de *tile* (*GrassTile, MontainTile, TreeTile, SoilTile și WaterTile*). În funcție de hartă, acel vector de *tile-uri* va conține *tile-uri* de acest tip.

GrassTile, MontainTile, TreeTile, SoilTile și WaterTile

Toate aceste clase extind clasa *Tile*, preluând comportamentul acelei clase sau putând să-l suprascrie folosind *@Override*. Constructorul acestora primește ca parametru un ID prin care se va putea identifica acel *tile* și apelează constructorul clasei de bază, adică al clasei *Tile*, care primește ca parametru un membru static al clasei Assets. Membrul static folosit este ales în funcție de tipul clasei respective, de exemplu pentru *GrassTile* se va folosi membrul static *grass.*

Assets

Conține câte un membru static de tip *BufferedImage* pentru fiecare *tile/dală*, chiar și pentru jucători/inamici. In acești membri sunt stocate imaginile, adică texturile acestora care vor fi desenate prin apelarea metodelor *Draw()* din fiecare clasa *tile* apelate din metoda *Draw()* din clasa *Game*. În acest exemplu este doar o imagine care conține toate texturile folosite, așadar pentru a putea instanța acest obiecte statice trebuie să decupăm fiecare textura din acea imagine. Pentru a face asta se folosește o instanță a clasei *SpriteSheet* în metoda *Init()*. Metoda folosită din *SpriteSheet* este *crop(x, y)*.

SpriteSheet

Constructorul acestei clase primește imaginea, iar clasa conține 2 membri constanți (*final* in Java) pentru înălțimea, respectiv lățimea texturilor (trebuie sa aibă toate aceleași dimensiuni). Metoda *crop(x, y)* primeste doi parametrii, *x* specifică pe ce coloană se află textura pe care dorim să o decupăm în imaginea cu toate

texturile, iar y specifică linia. Pentru a afla efectiv poziția în imagine se determină pixelii de unde încep texturile de interes prin înmulțirea liniei/coloanei cu înălțimea, respectiv lățimea texturilor. Astfel se obține colțul din stânga sus al texturii, iar pentru a afla celelalte colțuri se folosesc înălțimea, respectiv lățimea.

ImageLoader

Înainte de a extrage fiecare textură, imaginea cu toate texturile trebuie să fie citită din memorie, iar pentru asta se folosește clasa *ImageLoader* cu metoda statică *LoadImage(path)* care primește ca parametru calea către imagine în memoria calculatorului.

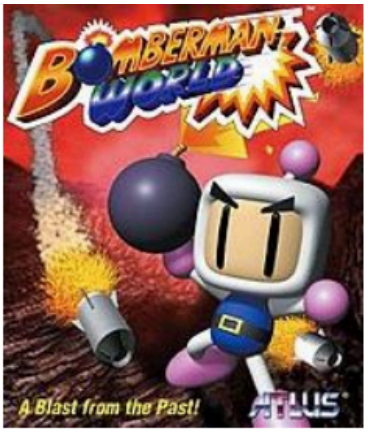
Anexa 1.

Titlu proiect (Bomberman)
Autori (Emil Popescu si Ion Popovici)
Grupa/Grupele(1200X respectiv 1200Y)

Documentarea va ține cont de aspect, respectare cerinte (inclusiv modul de scriere a documentației în Word: Justify mărime 12 la 1 rand pentru text, Titluri : mărime 14, Titlul documentului mărime 16; diacritice, aranjare in pagina, imaginile și tabelele –dacă există - aliniate central, etc)

O idee despre cum ar putea fi povestea jocului, care trebuie sa ne introducă în atmosfera jocului poate fi înțeleasă din textul de mai jos:

Jocul este situat undeva într-o galaxie cunoscută sub numele de Nebuloasa Bomber, de obicei pe planeta natală a lui Bomberman, "Planet Bomber". „Bomberman" (personajul omonim al jocului) a lucrat zi de zi făcând bombe într-o fabrică subterană a Imperiului Bungeling, dar a visat la libertate, așa că și-a planificat evadarea. După ce a auzit un zvon că roboții care ajung la suprafață devin oameni, el decide să scape. El a fost ajutat de singura abilitate pe care o cunoștea, fabricarea de bombe, folosind bombele pentru a distruge dușmanii care îi împiedicau evadarea și curățarea zidurilor blocate…



Nu uitați că în acest document trebuie să atingeți următoarele puncte:

- Proiectarea contextului și descriere detaliată
- O listă a sarcinilor săptămânale ale fiecărui membru al echipei -> GitHub Projects (task-urile vor fi corelate cu commit-urile de pe Git)
- Proiectarea sistemului și descriere detaliată -> Drawio / STARuml
- integrarea a minim 2 sabloane de proiectare (de exemplu, Singleton pentru gestionarea resurselor sau Factory pentru crearea inamicilor)
- corelarea sabloanelor de proiectare cu diagrama de proiect
- Proiectarea conținutului și descriere detaliată
- Proiectarea nivelurilor și descriere detaliată -> Tiled Map Editor
- Proiectul trebuie să aibă minim 3 niveluri

- Minim 2 game sprite-uri ce vor fi utilizate
- Proiectarea interfeței cu utilizatorul și descriere detaliată -> Figma / Canva
- Resursele biografice utilizate bibliografie in formatul IEEE